

Explainable AI

Why Explainability?

Complex AI models like deep learning, work like a "black box," where it is unclear how inputs translate to outputs.

What is Explainable AI?

- Field of AI focused on making AI systems' decisions understandable to humans.
- Aims to build methods that clarify how AI makes decisions.

Types of Models

- White-Box Models:
 - They are inherently interpretable because the decision-making process is clear and simple.
 - Examples: Linear Regression, Logistic Regression, Decision Trees etc.
 - White-box models provide better understanding but may sacrifice some accuracy.
 - White-box models usually run faster because they are simpler.
 - White-box AI is preferred when trust and transparency are critical (e.g., healthcare).
- Black-Box Models:
 - These are complex models which provide high accuracy but lack transparency.
 - Examples: Deep Neural Networks, Gradient Boosting, LLMs etc.
 - Black-box models often achieve higher performance due to their complexity.
 - Black-box AI excels in high-accuracy complex tasks like image recognition.

Model Interpretability

Method	Global	Local
Definition	How does the model behave on average across the entire dataset?	Why did the model make this specific prediction for this particular instance?
Precaution	Can miss nuances of individual predictions and may oversimplify complex local behaviours	Provides no summary of overall model behaviour; local explanations might not reflect global trends
Example	Global Surrogate Models, SHAP, PDP, Permutation Feature Importance etc.	Global Surrogate Models, SHAP, PDP, Permutation Feature Importance etc.
Use Case	Useful for detecting high-level biases and checking consistency with domain knowledge	Useful for detecting high-level biases and checking consistency with domain knowledge

Regulatory and Stakeholder Needs

- Regulatory Bodies: They require AI systems to be explainable, transparent, and auditable.
- Project Managers: They need explanations to monitor AI product behaviour and communicate AI decisions to stakeholders.
- Data Scientists: They require interpretability to debug models, detect biases and improve model robustness.
- End Users: They expect understandable reasons behind AI decisions, especially in sensitive domains like healthcare and finance.

Partial Dependency Plot (PDP)

- It is used to understand how a particular feature influences the predictions of an ML model on average (global interpretation).
- Example: Model predicting house prices - want to see how the price changes as the house size varies.
- A PDP shows the average predicted price for different house sizes, holding all other features constant.
- Helps answer questions such as "Does a larger house generally lead to a higher price?" or "Is the relationship linear or more complex?"
- Caveat: PDPs assume that the feature of interest is independent of other features – what if house size and neighbourhood are correlated?

How PDP Works?

1. Create the predictive model on the dataset.
2. Select the feature of interest for which you want to understand the effect on predictions.
3. Choose a sequence of values covering the range of the selected feature.
4. For each value in this sequence:
 - a. Replace the values of the selected feature in all instances of the dataset with the fixed value from the sequence while keeping other feature values as it is.
 - b. Use the trained model to predict outcomes for this modified dataset.
5. Average these predictions across all instances to get the expected prediction at that fixed feature value.
6. Plot these averages (y-axis) against the feature values (x-axis).

Individual Conditional Expectation

- It helps us understand how a model's prediction changes for each individual data point when we vary a feature (local interpretation).
- Unlike PDP, ICE plots display one line per instance.
- It is useful to detect interactions between features.
- **Example:** How the predicted price for each individual house changes if we hypothetically vary its size while keeping other features the same.
- Results in multiple lines—one for each house—allowing us to see patterns or differences between houses.

- If all lines are roughly parallel → feature's effect is consistent.
- If the lines cross or vary widely → feature's impact depends on other variables.

How ICE Works?

1. Create the predictive model on the dataset.
2. Select a single instance from the dataset.
3. Choose a sequence of values for the feature of interest.
4. For each value in this sequence:
 - a. Create a hypothetical instance by replacing the original feature value with the chosen values, keeping other feature values as it is.
 - b. Use the trained model to predict outcomes for these hypothetical instances.
5. Repeat the last step for every other instance in the dataset.
6. Plot all these individual curves together: predicted values (y-axis) against the feature values (x-axis).

Points to be Noted

- Graphs for both PDP and ICE are created using predicted data.
- For linear models, the PDP line will be linear.
- For linear models, the ICE lines for different instances will be parallel.
- Both methods assume feature independence from others.
- Correlated features can cause misleading interpretations.

Permutation Feature Importance

- It helps us understand which input features most influence a model's predictions.
- It measures the decrease in a model's performance when the values of a single feature are randomly shuffled (permuted).
- If a feature is important, shuffling its values should significantly degrade model performance.
- If a feature is unimportant, shuffling it will have little or no effect.
- It helps in global interpretation of a model.
- It also acts as a feature selection method.

Why is PFI Model Agnostic?

- Method does not depend on the internal structure of models (e.g., weights, splits)
- Only requires the ability of a model to make predictions and evaluate performance
- Works for linear models, tree-based models, neural networks, and even ensembles
- Easy to explain to stakeholders and reflects the overall contribution of a feature across the dataset

How PFI Works?

1. Train the model on the original dataset.
2. Compute baseline performance using a chosen metric.
3. For each feature:
 - a. Shuffle its values across all samples.
 - b. Typically, one random shuffle (permutation) is done per feature.
 - c. Compute the model's performance on this modified dataset.
 - d. Calculate the importance score (for RMSE, lower is better):
$$\text{Importance (f)} = \text{Baseline Performance} - \text{Performance after permutation of f}$$
4. If multiple random shuffles are done, average importance must be taken.
5. Using all possible permutations is computationally infeasible for large datasets.
6. If metric is accuracy, higher is better → importance formula will be reversed.

Pitfalls and Best Practices

- Data Leakage: A feature contains information that indirectly reveals the target.
- Correlated Features: When features are highly correlated, shuffling one does not fully remove its predictive power.
- Check for multicollinearity before interpreting permutation importance.
- Combine with other methods for robust interpretation.

Surrogate Models

- Complex ML models are hard to interpret.
- Surrogate models address this by providing interpretable, transparent approximations of these models.
- It uses the same input data.

- It tries to reproduce predictions of a complex model using a simpler model.
- The main objective is to give insight into the patterns of the complex model.
- It is not focused on achieving highest prediction accuracy.
- It needs no knowledge of complex model's internal structure.
- Local Surrogate Model: Approximates the predictions of a complex model for a single data instance.
- Global Surrogate Model: Approximates the predictions of a complex model across the entire dataset.

LIME Method

- Stands for local interpretable model-agnostic explanation
- Typically uses local linear surrogate modelling: A simple linear model trained to mimic the predictions of the complex model around a single instance
- Instead of explaining the entire model globally, explains why the model made a specific prediction
- Limitation: Computationally expensive for large datasets

How LIME Works?

1. Select an instance to explain.
2. Generate perturbed samples around this instance by slightly changing feature values.
3. Use the original model to predict the value of the output variable for all the new instances.
4. Calculate the distances of the new instances from the original instance.
 - a. Features are normalised before distance calculation
 - b. Binary features are one-hot encoded
 - c. Generally Euclidean distance is calculated
5. Assign weights to samples based on their distance from the original instance (closer samples get higher weight).
 - a. $\text{Weight} = e^{-\left(\frac{\text{Distance}}{\text{Kernel Width}}\right)^2}$
 - b. The kernel width (σ) is a hyperparameter; generally, it is equal to $\sqrt{\text{no. of features}} \times 0.75$
6. Fit a local weighted linear regression model for these new instances.
7. Interpret the coefficients of the linear model as local feature importance.

Other Considerations

- In Step 2, full Cartesian product of all variations isn't generally used as perturbed samples.
- LIME creates random perturbations for each feature independently.
- Non-linear surrogate models also exist: Useful for capturing non-linear relationships. Examples: Decision Tree and Neural Network Surrogate Models

Kernel Width

- Controls how quickly weights decrease with distance
$$\text{Weight} = e^{-\left(\frac{\text{Distance}}{\text{Kernel Width}}\right)^2}$$
- Small kernel width → focuses on very close neighbours → highly local explanation
- Large kernel width → includes more distant points → explanation is more global

SHAP Method

- Stands for Shapley Additive Explanation
- Concept grounded in cooperative game theory
- Uses Shapley Values: Each feature is treated as a “player” in a game where the goal is to predict the output
- Not a surrogate model

Context for SHAP

- Cooperative Game Theory
 - A group of players cooperate to achieve a common goal
 - Each player contributes differently to the outcome, so the challenge is to allocate credit/payoff fairly
 - In the context of ML: features = players, model prediction = payoff
- Shapley Value
 - Measures each player's contribution by its average marginal impact across all possible coalitions

- Similarly, SHAP computes each feature's contribution by averaging its marginal impact across all possible feature subsets

Types of SHAP

- Kernel SHAP: Model-agnostic but is computationally expensive
- Tree SHAP: Used for tree-based models (like XGBoost)
- Linear SHAP: Used for linear models
- Deep SHAP: Used for neural networks
- Permutation SHAP: Model-agnostic but slow

How Theoretical SHAP Works?

1. Create the predictive model on the dataset.
2. Select a specific instance from the dataset you want to study.
3. Generate all possible feature subsets for the model.
4. Calculate the baseline prediction (prediction for the null subset).
 - a. Baseline prediction = Expected value of the model's output over the entire dataset
= Represents the model's output if we know nothing about the specific instance
 - b. For classification, it is the average predicted probability for the class of interest across all observations.
 - c. For regression, it is the mean predicted value across all observations.
5. Calculate the model prediction for the specific instance (prediction for the full set of features).
6. Similarly, calculate the model prediction for the remaining subsets.
 - a. Whatever features are missing in the subset → Use the mean value
 - b. Whatever features are present in the subset → Use the value of the specific instance
 - c. Mathematically, $f(S)$ = Model prediction when only the features in the subset S are known
7. Select a feature i for which you want to calculate the Shapley value.
8. Find all feature subsets of the model that exclude the selected feature.
9. Assign weights for each such subset using the formula: $\text{Weight} = \frac{|S|!(|F|-|S|-1)!}{|F|!}$
 where, S = A particular subset which excludes the selected feature
 F = Set of all features
10. Calculate the marginal contribution of the selected feature.

- a. For all subsets in step 9, find the marginal contributions: $f(S \cup \{i\}) - f(S)$
 - i. Predicted value with that feature only – predicted value with no features OR
 - ii. Predicted value with several features (including the feature of interest) – predicted value with the same features (but excluding the feature of interest)
 - b. Weighted average of all such marginal contributions = Marginal contribution (Shapley value) of the selected feature i (ϕ_i)
11. Draw inferences based on the Shapley value.
- a. High SHAP value → Strong influence and vice versa
 - b. Positive value → Feature pulls prediction up and vice versa
12. Similarly, calculate the Shapley values for the other features.

How Kernel SHAP Works?

1. It is same as theoretical SHAP till Step 2.
2. From Step 3, subsets are binary encoded to create new datapoints: Use 1 if feature is included, 0 if not.
3. New datapoints are used to fit a weighted linear regression. Weights are calculated using Shapley formula:

$$w(S) = \frac{|F| - 1}{\binom{|F|}{|S|} \cdot |S| \cdot (|F| - |S|)}$$

where, $w(S)$ = Weight of subset S

F = Set of all features

4. Inference: Intercept denotes the baseline prediction. Coefficients denote the Shapley values for the features.

How Tree SHAP Works?

1. Define a tree model and select the instance to study.
2. Compute the baseline prediction: Take average of all predictions if the sample is uniformly distributed. If not, then weighted average needs to be calculated, i.e., $\sum(\text{leaf prediction} \times \text{leaf probability})$.
3. Calculate marginal contributions for each feature.
4. In case more features are present, weighted average is taken where the weights are calculated similar to theoretical SHAP.

What Happens in Practice?

- Enumerating all subsets is computationally expensive for large feature sets.
- Kernel SHAP uses sampling.
- Tree SHAP uses optimised algorithms (specifically dynamic programming).

Features of SHAP

- Ensures Additivity: Sum of Shapley values of all features + base prediction = model prediction.
- Guarantees Consistency: If a feature's contribution increases in another model, its SHAP value cannot decrease.
- Advantages:
 - Theoretically sound (as based on game theory)
 - Works for both global and local explanations
- Disadvantages:
 - Computationally expensive for large feature sets
 - Requires careful interpretation in correlated feature scenarios

DeepLIFT Method

- It is tailored for deep neural networks.
- Reference Activation: For every neuron in the network, DeepLIFT computes its activation for a real input and compares it against a "reference" input.
- Contribution Scores: Difference between actual and reference activations is propagated backward through the network, layer by layer.
- Contributions are calculated for each neuron: At every layer, the contribution is split proportionally among the incoming neurons based on their weights and activation.
- Backpropagation continues all the way to the input layer: Scores for each input feature show how much it contributed to the prediction.

Comparison of XAI Libraries

Parameter	LIME	SHAP	DeepLIFT
Explanation Scope	Suitable for Local	Local and Global (by local aggregation)	Suitable for local and layer-wise
Model Type	Any (Agnostic)	Any (Agnostic)	Deep Learning (compatible with Keras/Tensorflow)
Strengths	Simple and intuitive	Consistent, reliable, rich visuals	Handles non-linearities better
Limitations	Local only, unstable	Computationally intensive	Deep learning only
Sensitive To	Kernel Width, Sampling	Dataset used for baseline calculations	Reference Input
Speed	Fast	Slower on large datasets (specially Kernel SHAP)	Fast (but only for deep models)
Use Cases	Fast explanation of individual instances	Model debugging, model fairness studies	Deep neural networks for understanding neuron/layer contributions
Examples	Tabular data, text and image data	Tree-based models, neural networks	Medical imaging and computer vision

Disclaimer: All content and material on the upGrad website is copyrighted material, either belonging to upGrad or its bona fide contributors and is purely for the dissemination of education. You are permitted to access, print, and download extracts from this site purely for your own education only and on the following basis:

- You can download this document from the website for self-use only.
- Any copies of this document, in part or full, saved to disk or to any other storage medium, may only be used for subsequent, self-viewing purposes or to print an individual extract or copy for non-commercial personal use only.
- Any further dissemination, distribution, reproduction or copying of the content of the document herein or the uploading thereof on other websites, or use of the content for any other commercial/unauthorized purposes in any way which could infringe the intellectual property rights of upGrad or its contributors, is strictly prohibited.
- No graphics, images, or photographs from any accompanying text in this document will be used separately for unauthorized purposes.
- No material in this document will be modified, adapted, or altered in any way.
- No part of this document or upGrad content may be reproduced or stored on any other website or included in any public or private electronic retrieval system or service without prior written permission from upGrad.
- Any rights not expressly granted in these terms are reserved.